

Security Audit

Quantum Pools (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Deployed Contracts	8
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	27
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Quantum Pools partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Quantum Pools is a custodial, per-position concentrated-liquidity management protocol. A user deposits tokens into an approved Uniswap-v3-shaped pool; the vault holds the resulting position NFT while recording the depositor as the position owner. An authorized keeper may advance an out-of-range state machine and recenter positions, while users retain owner-gated withdrawal rights. The reviewed VenueAdapter library supports Uniswap V3, PancakeSwap V3 and Aerodrome Slipstream pools. Both deployments listed in this report use Uniswap V3.

Project Name: Quantum Pools

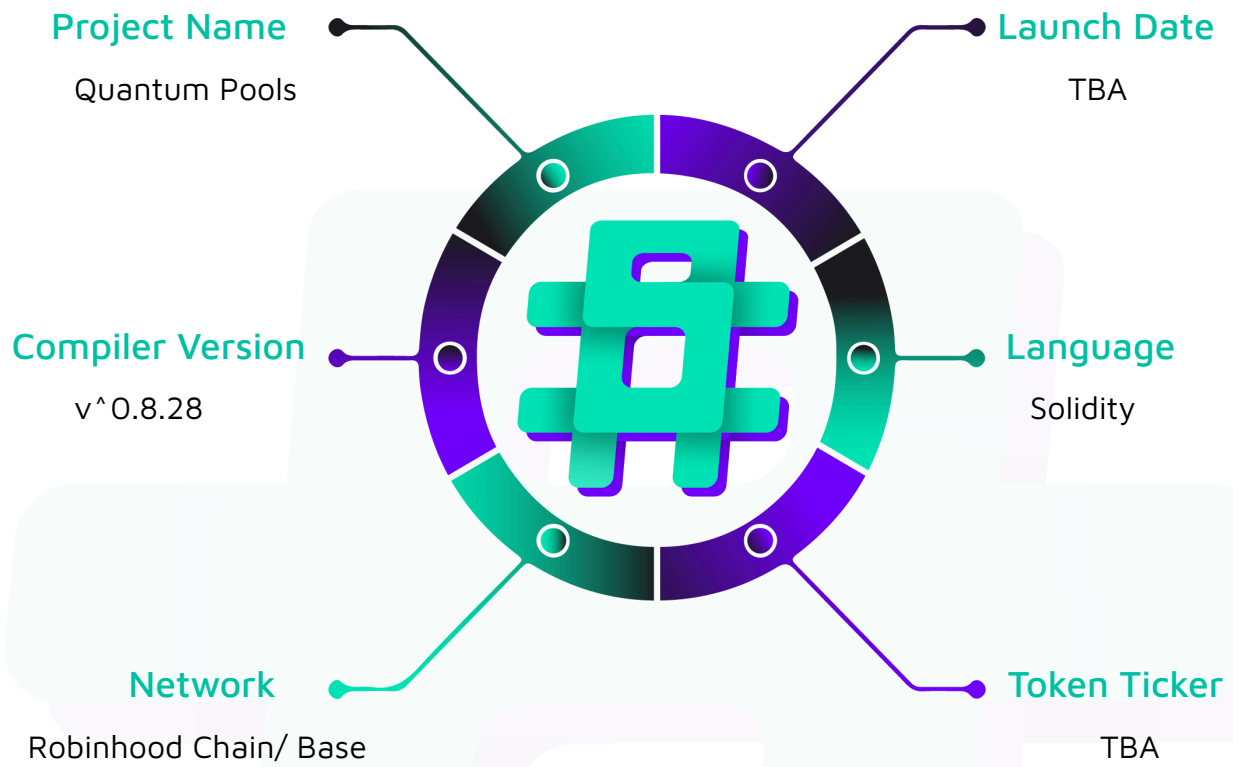
Project Type: Defi, concentrated-liquidity vault, custodial NFT position manager

Compiler Version: ^0.8.28

Website: <https://app.quantumpools.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Quantum Pools project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Quantum Pools Smart Contracts
Network	Robinhood Chain / Base
Language	Solidity
Audit Date	September, 2026
Contract 1	QuantumVaultV2.sol
Contract 2	PositionQueue.sol
Contract 3	VaultMath.sol
Contract 4	VenueAdapter.sol
Contract 5	FeeRouter.sol
Deployment script	DeployV2.s.sol
Audited GitHub Commit Hash	6ccce2e
Fix Review GitHub Commit Hash	f2ca3d2

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Deployed Contracts

Contract	Commit	Network	Address
QuantumVaultV2 (live)	f2ca3d2(Fix Review)	Robinhood Chain	0x5433f385F538Fa11b8A25B82230a306d25dF8EB3
QuantumVaultV2 (live)	f2ca3d2(Fix Review)	Base	0x9597f68203d1bc85744A5Fc11D890aA7777e386B
QuantumVaultV2 (superseded, being paused)	6ccce2e (Audited)	Robinhood Chain	0x07c688E3C1d606edc7Fe428862C08c3FA9aAEb3B
FeeRouter	f2ca3d2	Robinhood Chain, Base	0xF383d5408Aa36005B59952a05cE84D098e54C9f2
PositionQueue	f2ca3d2	Robinhood Chain, Base	0xC6Fe2Ac3b6Ba2b089F263324634aF37E45E3993F
VaultMath	f2ca3d2	Robinhood Chain, Base	0x9b6D35FA063ddB11d7e59ef00454C9134e211940
VenueAdapter	f2ca3d2	Robinhood Chain, Base	0xb4ACC0209fC349EC509362CcA7d5fC4C5719dc60

Note: Hashlock verified that the live QuantumVaultV2 deployments on Robinhood Chain and Base match the Fix Review commit (f2ca3d2), differing only in constructor immutables (each chain's Uniswap V3 position manager and factory), and that both are linked to the library addresses listed above. The superseded vault deployed from the Audited commit (6ccce2e) does not include the fixes described in this report.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
QuantumVaultV2.sol - Allows users to open concentrated-liquidity positions in owner-approved pools. - Keeps the position NFT in vault custody while recording the depositor as the position owner. - Allows position owners to fully or partially withdraw without being blocked by the protocol pause. - Allows owners to queue capital for the next recenter and cancel the queue before investment. - Allows owners to queue width and delay changes for application at the next recenter. - Requires automated rebalancing only after the position has remained continuously out of range for the configured delay. - Charges the performance fee only on collected trading fees, not principal.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Quantum Pools, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Quantum Pools smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] contracts/src/QuantumVaultV2.sol#_recenter - Queued strategy is deleted before the successor is initialized

Description

A pending width or delay is stored on the current position. Recenter reads the queued width for the replacement range, then deletes the pending entry during the exit, before the successor strategy is initialized.

Vulnerability Details

`_rebalanceAuto` loads the queued width and uses it to derive the range for this mint. The comment says the entry is left in storage so `_recenter` can apply it later.

```
PositionQueue.PendingStrategy memory q = pendingStrategies[tokenId];
int24 pendingWidth = q.setWidth ? q.rangeWidthTicks :
strategies[tokenId].rangeWidthTicks;
if (pendingWidth > 0) width = pendingWidth;

(int24 lower, int24 upper) =
    VaultMath.recenterRange(spot, width, isToken0,
IUniswapV3Pool(pool).tickSpacing());
if (lower == oldLower && upper == oldUpper) revert NoChange();
newTokenId = _recenter(tokenId, RecenterPlan(lower, upper, isToken0), rp);
```

`_recenter` snapshots the existing strategy, then calls `_exitLiquidity`. That path reaches `_removePosition`, which deletes `pendingStrategies[tokenId]` before `applyStrategy` runs.

```
Strategy memory s = strategies[tokenId];
if (s.compoundBps == 0 && s.feeMode == 0 && !s.autoSnuggle && s.rangeWidthTicks == 0)
{
    s.feeMode = uint8(FeeRouter.Mode.SPLIT);
```

```

    s.compoundBps = DEFAULT_COMPOUND_BPS;
}
uint256 routeTarget = _liveTarget(feeRouteTarget[tokenId]);

(uint256 add0, uint256 add1) = PositionQueue.consume(pendingDeposits, tokenId, 0, 0);
(uint256 q0, uint256 q1) = (add0, add1);

FeeAccounting memory fa = feeAccounting[tokenId];
{
    (uint256 c0, uint256 c1) = _collectFees(tokenId);
    fa.feesEarnedCumulative0 += c0;
    fa.feesEarnedCumulative1 += c1;
    (uint256 bal0, uint256 bal1) =
        _exitLiquidity(tokenId, rp.decreaseAmount0Min, rp.decreaseAmount1Min,
rp.deadline);
    (uint256 reinv0, uint256 reinv1) = _settleAndOffset(tokenId, p, pos.owner, fa,
bal0, bal1);
    (reinv0, reinv1) = _routeFees(s, p, tokenId, routeTarget, pos.owner, reinv0,
reinv1, plan.isToken0Side);
    add0 += bal0 + reinv0;
    add1 += bal1 + reinv1;
}
fa.principal0 += q0;
fa.principal1 += q1;
(s.rangeWidthTicks, s.rebalanceDelay) =
    PositionQueue.applyStrategy(pendingStrategies, tokenId, s.rangeWidthTicks,
s.rebalanceDelay);

```

```

delete pendingStrategies[tokenId];

```

applyStrategy then reads an empty slot. Both flags are false, so it returns the snapshotted width and delay unchanged. _finishRecenter stores that old strategy on the new token id.

```

PendingStrategy memory pending = q[tokenId];
if (!pending.setWidth && !pending.setDelay) return (width, delay);

```

The queued width therefore affects only the position minted in that recenter. The successor keeps the previous width, and the following recenter returns to it. The queued delay is never copied, because delay is not an input to the immediate range calculation and the entry is already gone. Both outcomes share this deletion.

Impact

An owner's ordinary strategy update appears to succeed, then either lasts for one cycle or never takes effect. Later keeper recenters use the stale width, or they fire on the old delay. For concentrated liquidity, the wrong range converts inventory and exposes the position to arbitrage. The loss depends on price movement and position size.

Recommendation

Snapshot the full pending strategy before any cleanup:

```
PendingStrategy memory pending = pendingStrategies[tokenId];
```

Use that snapshot to derive the replacement range, to set every strategy field on the successor, and to emit the applied configuration. Delete the old pending entry only after the successor is initialized.

Status

Resolved

[M-02] contracts/src/QuantumVaultV2.sol#_keeperAction - The out-of-range delay is not measured over one continuous excursion

Description

The position is supposed to stay out of range for `rebalanceDelay` before a recenter. An in-range observation during the reset lock does not end that episode, so the old `outOfRangeSince` keeps counting.

Vulnerability Details

Action 1 records the start of tracking:

```
if (s.outOfRangeSince != 0) revert AlreadyTracking();
if (!oor) revert NotOutOfRange();
s.outOfRangeSince = uint40(block.timestamp);
```

Action 2 recenters once that timestamp plus `rebalanceDelay` has elapsed, provided the position is out of range now:

```
if (s.outOfRangeSince == 0) revert NotTracking();
if (!oor) revert NotOutOfRange();
if (block.timestamp < uint256(s.outOfRangeSince) + s.rebalanceDelay) revert
DelayNotMet();
```

Action 3 is the only keeper path that clears the timestamp, and it refuses to do so until five minutes have passed:

```
uint256 public constant MIN_OOR_TRACKING_INTERVAL = 5 minutes;

if (oor) revert StillOutOfRange();
if (s.outOfRangeSince == 0) revert NotTracking();
if (block.timestamp < uint256(s.outOfRangeSince) + MIN_OOR_TRACKING_INTERVAL) revert
TrackingIntervalNotMet();
s.outOfRangeSince = 0;
```

A failing sequence with a short delay:

```
t = 0:00   Position goes out of range. outOfRangeSince is recorded.
t = 4:00   Position returns in range. Action 3 reverts TrackingIntervalNotMet.
t = 5:00   Position leaves the range again.
```

`t = 5:01` Action 2 treats the full delay as elapsed.

The second excursion has lasted seconds. It inherits the first episode because the in-range observation couldn't clear the clock. There is no minimum `rebalanceDelay`, so a delay on the order of this five-minute lock is a valid configuration. A longer delay is shortened by the in-range gap that occurred while the reset was refused, which is bounded by that lock plus keeper timing. A reverted action 3 also marks the position bad in the keeper and backs off further attempts, so the reset can be skipped again after the lock expires.

Impact

The position can be recentered before it has been continuously out of range for the configured delay. Ordinary price movement can produce the in-range observation. A caller who can move the pool tick can force the same sequence.

Recommendation

An in-range observation must clear the active out-of-range episode.

Status

Resolved

Low

[L-01] contracts/src/QuantumVaultV2.sol#setStrategy - A valid zero fee configuration collides with the unset representation

Description

COMPOUND_ALL with 0 bps, and SPLIT with 0 bps, are valid stored policies. The same zeros are also the marker that no fee policy was configured, so a later strategy edit or recenter replaces the owner's choice with the default split.

Vulnerability Details

COMPOUND_ALL is enum value 0. FeeRouter.validate accepts any compoundBps up to 10_000 for every mode except ROUTE, including 0.

```
enum Mode {
    COMPOUND_ALL,
    WALLET_ALL,
    SPLIT,
    MATCHING,
    ROUTE
}
```

setStrategy treats the all-zero strategy as "never configured" and writes SPLIT. It also rewrites an explicit SPLIT whose compoundBps is 0 to DEFAULT_COMPOUND_BPS (5000), even when a width is already stored:

```
uint8 mode = prev.compoundBps == 0 && prev.feeMode == 0 && !prev.autoSnuggle &&
prev.rangeWidthTicks == 0
    ? uint8(FeeRouter.Mode.SPLIT)
    : prev.feeMode;
uint16 cbps = prev.compoundBps == 0 && mode == uint8(FeeRouter.Mode.SPLIT) ?
DEFAULT_COMPOUND_BPS : prev.compoundBps;
```

_recenter repeats the all-zero check and carries the rewritten strategy onto the successor:

```
if (s.compoundBps == 0 && s.feeMode == 0 && !s.autoSnuggle && s.rangeWidthTicks == 0)
```

```
{  
    s.feeMode = uint8(FeeRouter.Mode.SPLIT);  
    s.compoundBps = DEFAULT_COMPOUND_BPS;  
}
```

COMPOUND_ALL ignores compoundBps, so mode 0 with 0 bps is the natural encoding of “compound everything.” On a fresh position, rangeWidthTicks and autoSnuggle are still zero, so that policy is indistinguishable from unset. The SPLIT / 0 bps rewrite doesn't need a zero width: every later setStrategy turns “reinvest 0%” into “reinvest 50%.”

Impact

Fee proceeds are routed differently from the policy the owner stored. SPLIT at 50% pays half of the matching side to the wallet and compounds the rest. COMPOUND_ALL compounds both sides. SPLIT at 0 bps pays the matching side out. The change happens on a later strategy edit or manual recenter, with no fee-policy update requested.

Recommendation

Add an explicit configured flag, or give the enum a zero-valued UNSET and move every valid mode to a nonzero value. Treat SPLIT with 0 bps as a real policy in setStrategy.

Status

Resolved

[L-02] contracts/foundry.toml - The default build profile hard-links privileged libraries

Description

The default Foundry profile pins the four external libraries to fixed addresses. DeployV2 states that Forge deploys fresh library instances and links them as part of the script. A build under the default profile resolves those links at compile time, so the vault bytecode keeps the pinned addresses.

Vulnerability Details

```
[profile.default]
libraries = [

    "src/libraries/FeeRouter.sol:FeeRouter:0xF383d5408Aa36005B59952a05cE84D098e54C9f2",

    "src/libraries/PositionQueue.sol:PositionQueue:0xC6Fe2Ac3b6Ba2b089F263324634aF37E45E3993F",

    "src/libraries/VaultMath.sol:VaultMath:0x9b6D35FA063ddb11d7e59ef00454C9134e211940",

    "src/libraries/VenueAdapter.sol:VenueAdapter:0xb4ACC0209fC349EC509362CcA7d5fC4C5719dc60",

]
```

The script checks the position manager, then deploys the vault:

```
require(npm.code.length > 0, "NPM has no code on this chain");
address factory = INonfungiblePositionManager(npm).factory();
require(factory.code.length > 0, "NPM.factory() is not a contract");
require(treasury != address(0), "TREASURY unset");
require(feeBps <= 3_000, "FEE_BPS above the contract's hard cap");
if (requireTimelock) {
    require(owner.code.length > 0, "OWNER must be the timelock contract (set REQUIRE_TIMELOCK=false for a dry run)");
}
uint256 runtime = vm.getCode("QuantumVaultV2.sol:QuantumVaultV2").length;
console2.log("creation code bytes:", runtime);
```

```
vm.startBroadcast();  
vault = new QuantumVaultV2(owner, npm, treasury, feeBps);  
vm.stopBroadcast();
```

forge script uses the default profile unless another profile is selected. Fixed library addresses are substituted into the vault bytecode when it is built. The script's preflight checks that the position manager has code. It does not check `address.code.length` or the runtime hash of `FeeRouter`, `PositionQueue`, `VaultMath`, or `VenueAdapter`. Those libraries are public, so the vault reaches them by `DELEGATECALL` and they execute with the vault's storage and token authority.

Impact

This is a deployment-configuration defect. A pure contest that trusts the already-broadcast bytecode can treat the live vault as out of scope, because that bytecode's link set matches the pin. A new deployment under the default profile can still come up linked to the wrong targets and look successful. Because the targets run by `delegatecall`, incompatible or attacker-controlled code has the vault's authority over storage and assets.

Recommendation

Remove production library addresses from the default profile. Use explicit chain-specific profiles. Link the vault to the library addresses the script just deployed.

Status

Resolved

[L-03] contracts/src/QuantumVaultV2.sol#_distributeCut - Repeated small collections permanently avoid protocol fees

Description

Each collection rounds the protocol fee down independently, then marks the entire collected amount as already taxed. The discarded fraction is never charged on a later collection.

Vulnerability Details

`_takeFee` advances the high-water mark by the full taxable slice, and only then computes the cut. `_settleAndOffset` does the same on the recenter path.

```
fa.feesTaxed0 += taxable0;
fa.feesTaxed1 += taxable1;
(cut0, cut1) = _tax(tokenId, feeOwner, token0, token1, taxable0, taxable1);
```

```
uint256 u0 = fa.feesEarnedCumulative0 - fa.feesTaxed0;
uint256 u1 = fa.feesEarnedCumulative1 - fa.feesTaxed1;
fa.feesTaxed0 += u0;
fa.feesTaxed1 += u1;
```

The cut itself is integer division:

```
uint256 total = (taxable * bps) / BPS;
```

BPS is 10_000. For the live 1500 bps rate, any slice below 7 raw units produces a zero cut, and that slice is still checkpointed. Splitting one collection therefore yields:

```
sum(floor(slice[i] * bps / 10_000)) < floor(sum(slice[i]) * bps / 10_000)
```

At 10% (1000 bps), one collection of 90 units charges 9. Ten collections of 9 units charge 0, and the 90 units are marked processed.

The position owner chooses when to harvest; keeper snuggles of opted-in, out-of-range positions are also separate collections.

Impact

Protocol fee revenue on those slices is skipped.

Recommendation

Carry the remainder forward:

```
numerator = newlyCollected * feeBps + previousRemainder;  
protocolFee = numerator / BPS;  
newRemainder = numerator % BPS;
```

A second valid approach is to leave the untaxed dust uncheckpointed, so a later collection still includes it in the taxable base. Add a regression test that compares one aggregate collection with many collections just below the rounding threshold.

Status

Resolved

QA

[Q-01] keeper-worker/src/core.js#finish - Failed primary iterations refresh the heartbeat and suppress the backup

Description

The primary keeper publishes a fresh heartbeat after a handled failure. The backup stands down on the age of that heartbeat, so a live but non-functional primary prevents failover indefinitely.

Vulnerability Details

Handled failures return through `finish()`, including a missing vault, a wrong ABI, and a keeper that is not authorized:

```
if (!code || code.length < 400) return finish({ error: `no vault code at ${vault}`
});
if (!code.includes(sel)) {
  return finish({ error: `VAULT_VERSION=${version} but ${vault} has no
executeKeeperAction selector 0x${sel} - wrong version for this vault` });
}
```

```
if (!isKeeper) return finish({ error: `not authorized: ${account.address} is not in
authorizedKeepers` });
```

`finish` always writes the heartbeat, including when `result.error` is set:

```
await kv.put('heartbeat', JSON.stringify(beat), { expirationTtl: 86400 });
if (result.error) await kv.put('last_error', JSON.stringify(beat), { expirationTtl:
604800 });
```

The public status age is `now - beat.epoch`. It does not depend on whether the cycle succeeded. The backup exits when that age is under `KEEPER_STALE_SEC` (420 seconds by default):

```
const age = await primaryAgeSec();
if (age !== null && age < STALE_SEC) {
```

```
log(`primary healthy (heartbeat ${age}s old) - standing down`);  
process.exit(0);  
}
```

Impact

Positions can stay outside their ranges during volatile conditions while a healthy backup is available and idle. Monitoring still reports the primary as alive. For an automated liquidity manager this is a loss of availability and of the management the depositor configured.

Recommendation

Publish `lastAttempt` and `lastSuccessfulCycle` separately. Failover must follow `lastSuccessfulCycle`.

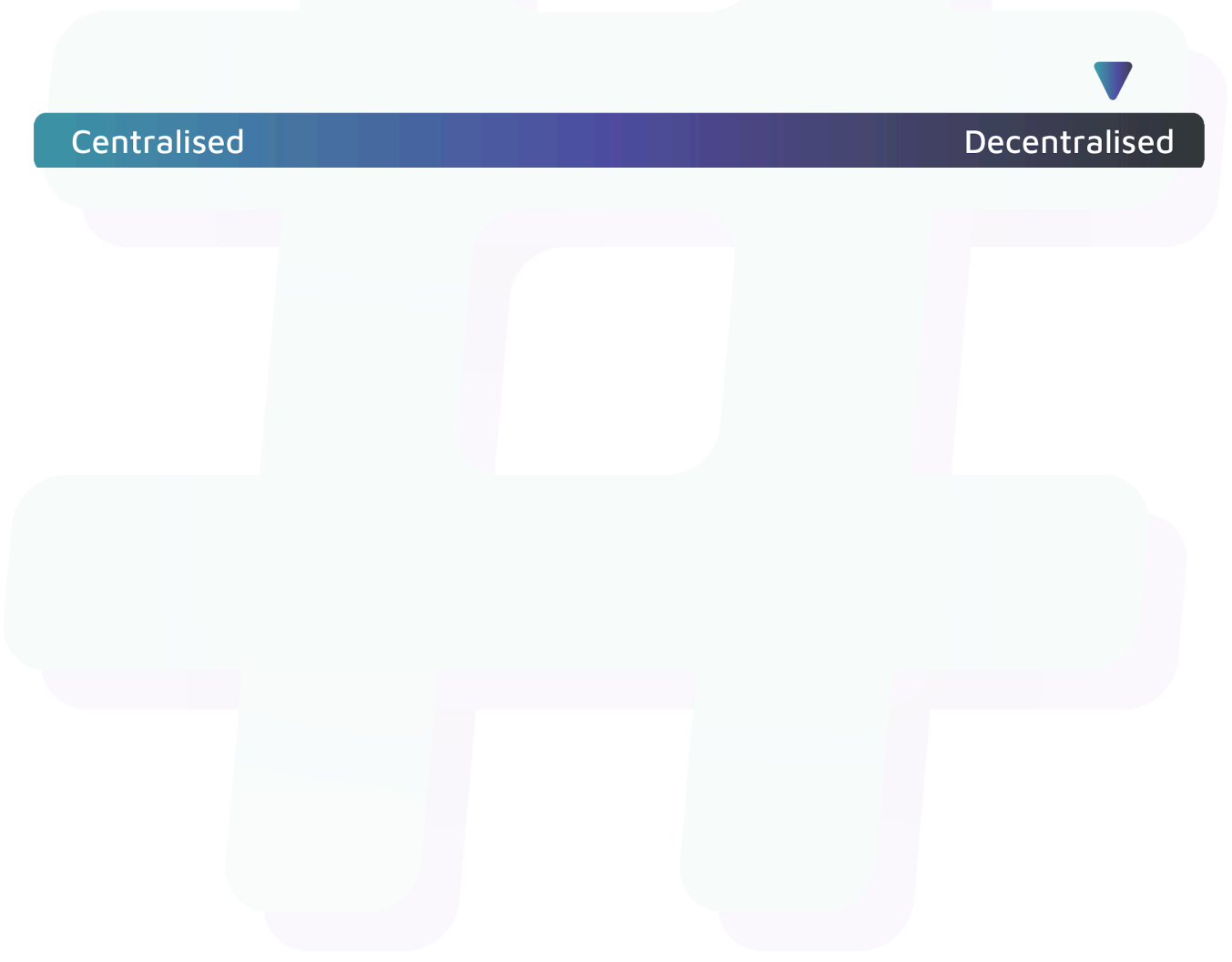
Status

Resolved

Centralisation

The Quantum Pools protocol values security and utility over decentralisation.

All owner functions on both live vaults are controlled by a 48-hour TimelockController (Robinhood Chain: `0x41D34F171bBE138F8eA297f95e5B6B6F298a27AC`; Base: `0xE0db1d799dB2FF5aC539B95C3299c8AF060B4088`), whose only proposer, executor and canceller is a 2-of-2 Safe (`0x00B3DCd95dDb5D1454dcbbfA6805e17f82244Ac1`) held by the two cofounders. The timelock gives users time to review and react to queued changes, while control of the protocol remains with the core team.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Quantum Pools project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.